Practical No.1

Aim: To study and implement recursive algorithms using C++ by solving:

1. Factorial Computation
2. Tower of Hanoi problem

Software Requirements: Visual Studio Code (Text Editor / IDE), MinGW (Minimalist GNU for Windows) for Windows (Compiler), Dev C++, Turbo C++

Objectives:

1. To understand the concept of recursion and recursive function calls in C++.
2. To implement a recursive algorithm for factorial computation.
3. To implement the Tower of Hanoi problem using recursion.

Theory:

Recursion:

Recursion is a programming technique in which a function calls itself to solve a problem. A recursive solution breaks a problem into smaller sub-problems of the same type until a base condition is reached. The base condition stops further recursive calls and prevents infinite recursion.

Every recursive algorithm consists of:

1. **Base Case** – the condition where recursion stops.
2. **Recursive Case** – the part where the function calls itself with a smaller input.

1. Factorial Computation Using Recursion

The factorial of a non-negative integer n is defined as:

$$n! = n \times (n - 1) \times (n - 2) \times \cdots \times 1$$

By definition:

- 0! = 1
- 1! = 1

Recursive Definition

$$n! = n \times (n - 1)!$$

In recursion:

- **Base Case:** If n = 0 or n = 1, return 1.
- **Recursive Case:** Multiply n by the factorial of n-1.

This approach clearly demonstrates how recursion reduces a problem step by step until it reaches the base case.

2. Tower of Hanoi Problem

The **Tower of Hanoi** is a classic problem used to explain recursion. It consists of:

- Three rods: **Source (A)**, **Auxiliary (B)**, and **Destination (C)**
- n disks of different sizes placed on the source rod

Rules

1. Only one disk can be moved at a time.
2. A disk can be placed only on top of a larger disk.
3. All disks must be moved from the source rod to the destination rod.

Recursive Solution

To move n disks:

1. Move n−1 disks from Source to Auxiliary using Destination as helper.
2. Move the largest disk from Source to Destination.
3. Move n−1 disks from Auxiliary to Destination using Source as helper.

Base Case

- If n = 1, move the disk directly from Source to Destination.

The Tower of Hanoi problem demonstrates how a complex problem can be solved efficiently by dividing it into smaller sub-problems using recursion.

Program: Factorial Computation (Recursive):

```
#include <iostream>
using namespace std;

int factorial(int n)
{
   if (n == 0)
      return 1;
   else
      return n * factorial(n - 1);
}

int main()
{
   int num;
   cout << "Enter a number: ";
   cin >> num;

   cout << "Factorial of " << num << " is: " << factorial(num);

   return 0;
}
```

Output:

Enter a number: 5
Factorial of 5 is: 120

Program: Tower of Hanoi (Simple Recursive)

```
#include <iostream>
using namespace std;
void towerOfHanoi(int n, char from, char to, char aux)
{
```

```
    if (n == 1)
    {
        cout << "Move disk 1 from " << from << " to " << to << endl;
        return;
    }
    towerOfHanoi(n - 1, from, aux, to);
    cout << "Move disk " << n << " from " << from << " to " << to << endl;
    towerOfHanoi(n - 1, aux, to, from);
}
int main()
{
    int n;
    cout << "Enter number of disks: ";
    cin >> n;
    cout << "Steps to solve Tower of Hanoi:\n";
    towerOfHanoi(n, 'A', 'C', 'B');
    return 0;
}
```

Output (for 3 disks)

Enter number of disks: 3
Steps to solve Tower of Hanoi:
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

Conclusion:

In this experiment, recursive algorithms were studied and implemented using C++. The factorial computation demonstrated how a problem can be solved by reducing it to a smaller instance of the same problem using a base condition. The Tower of Hanoi problem illustrated the power of recursion in solving complex problems by dividing them into simpler sub-problems.

Assignment / Viva Questions:

1. What is recursion? Why is a base case important in a recursive function?
2. How many rods are used in the Tower of Hanoi problem?
3. Compare recursion and iteration.
4. State any two advantages & advantages of recursion.
5. Where is recursion commonly used in real-life applications?

Practical No.2

Aim: To study and implement non-recursive (iterative) sorting algorithms—Bubble Sort, Insertion Sort, and Selection Sort—using C++.

Software Requirements: Visual Studio Code (Text Editor / IDE), MinGW (Minimalist GNU for Windows) for Windows (Compiler), Dev C++, Turbo C++

Objectives:

1. To understand the concept of non-recursive (iterative) algorithms.
2. To study the working principles of Bubble Sort, Insertion Sort, and Selection Sort.
3. To implement sorting algorithms using loops in C++.

Theory:

Non-Recursive Algorithms

Non-recursive algorithms solve problems using iterative constructs such as for, while, or do-while loops instead of function calls. These algorithms are generally easier to understand, consume less memory, and avoid the overhead of recursive calls.

Sorting is a fundamental operation in computer science that arranges data in a specific order (ascending or descending).

1. Bubble Sort

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. After each pass, the largest element moves to its correct position.

Time Complexity:

- Best Case: $O(n)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Algorithm: Bubble Sort

Step 1: Start
Step 2: Read n (number of elements)
Step 3: Read array elements
Step 4: Repeat for i = 0 to n − 2
  a) Repeat for j = 0 to n − i − 2
  b) If a[j] > a[j+1], swap them
Step 5: Display the sorted array
Step 6: Stop

2. Insertion Sort

Insertion Sort works like sorting playing cards in hand. It picks an element and inserts it into its correct position in the already sorted part of the array.

Time Complexity:

- Best Case: O(n)
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Algorithm: Insertion Sort

Step 1: Start
Step 2: Read `n` (number of elements)
Step 3: Read array elements
Step 4: For `i = 1` to `n - 1`
  a) Set `key = a[i]`
  b) Set `j = i - 1`
  c) While `j ≥ 0` and `a[j] > key`
    Shift `a[j]` to `a[j+1]`
    Decrement `j`
  d) Insert `key` at position `j + 1`
Step 5: Display the sorted array
Step 6: Stop

3. Selection Sort

Selection Sort repeatedly selects the smallest element from the unsorted portion and places it at the beginning.

Time Complexity:

- Best, Average, Worst Case: $O(n^2)$

Algorithm: Selection Sort

Step 1: Start
Step 2: Read `n` (number of elements)
Step 3: Read array elements
Step 4: For `i = 0` to `n - 2`
  a) Set `min = i`
  b) For `j = i + 1` to `n - 1`
    If `a[j] < a[min]`, set `min = j`
  c) Swap `a[i]` and `a[min]`
Step 5: Display the sorted array
Step 6: Stop

Bubble Sort Program:

```
#include <iostream>
```

```
using namespace std;
int main() {
int n, a[50];
cout << "Enter number of elements: ";
cin >> n;
cout << "Enter elements:\n";
for(int i = 0; i < n; i++)
cin >> a[i];
for(int i = 0; i < n - 1; i++) {
for(int j = 0; j < n - i - 1; j++) {
if(a[j] > a[j + 1]) {
int temp = a[j];
a[j] = a[j + 1];
a[j + 1] = temp;
}
}
}
cout << "Sorted array:\n";
for(int i = 0; i < n; i++)
cout << a[i] << " ";
return 0;
}
```

Output: Bubble Sort

Enter number of elements: 5

Enter elements:

57

37

89

24

60

Sorted array:

24 37 57 60 89

Insertion Sort Program:

```
#include <iostream>
using namespace std;
int main() {
int n, a[50];
cout << "Enter number of elements: ";
cin >> n;
cout << "Enter elements:\n";
for(int i = 0; i < n; i++)
cin >> a[i];
for(int i = 1; i < n; i++) {
int key = a[i];
int j = i - 1;
while(j >= 0 && a[j] > key) {
a[j + 1] = a[j];
j--;
}
a[j + 1] = key;
}
cout << "Sorted array:\n";
for(int i = 0; i < n; i++)
cout << a[i] << " ";
return 0;
}
```

Output: Insertion Sort

```
Enter number of elements: 5
Enter elements:
39
59
24
11
64
Sorted array:
11 24 39 59 64
```

Selection Sort Program:

```
#include <iostream>
using namespace std;
int main() {
int n, a[50];
cout << "Enter number of elements: ";
cin >> n;
cout << "Enter elements:\n";
for(int i = 0; i < n; i++)
cin >> a[i];
for(int i = 0; i < n - 1; i++) {
int min = i;
for(int j = i + 1; j < n; j++) {
if(a[j] < a[min])
min = j;
}
int temp = a[i];
a[i] = a[min];
a[min] = temp;
}
cout << "Sorted array:\n";
for(int i = 0; i < n; i++)
cout << a[i] << " ";
return 0;
}
```

Output: Selection Sort

```
Enter number of elements: 5
Enter elements:
12
42
15
67
04
```

Sorted array:

4 12 15 42 67

Conclusion:

In this experiment, non-recursive sorting algorithms were successfully studied and implemented using C++. These algorithms help in understanding basic sorting concepts, loop-based problem solving, and time complexity analysis.

Assignment/Viva Questions:

1. Define sorting. Why is sorting important in data processing?
2. What is adaptive sorting? Which of above algorithm is adaptive?
3. Compare Bubble Sort, Insertion Sort, and Selection Sort based on: Time complexity, Space complexity, Stability, Adaptiveness
4. Which sorting algorithm would you choose for a nearly sorted list and why?
5. Give real-life applications of each algorithm.

Practical No.3

Aim: To study and implement Binary Search, Merge Sort, Quick Sort.

Software Requirements: Visual Studio Code (Text Editor / IDE), MinGW (Minimalist GNU for Windows) for Windows (Compiler), Dev C++, Turbo C++

Objectives:

6. To study and implement Binary Search
7. To understand Divide and Conquer strategy using Merge Sort.
8. To understand partition-based sorting using Quick Sort.

Theory:

1. Binary Search:

Binary Search is an efficient searching algorithm that works on **sorted arrays**.
It repeatedly divides the search interval into two halves and compares the target element with the middle element of the array.

Working Principle

- If the middle element equals the key → element found
- If key < middle element → search left subarray
- If key > middle element → search right subarray
- Repeat until element is found or interval becomes empty

Time Complexity

- Best Case: **O(1)**
- Average Case: **O(log n)**
- Worst Case: **O(log n)**

Algorithm

1. Start
2. Read number of elements n
3. Read sorted array arr[]
4. Read element to be searched key
5. Set low = 0, high = n-1
6. While low ≤ high
    - Compute mid = (low + high) / 2
    - If arr[mid] == key, print position and stop
    - Else if key < arr[mid], set high = mid - 1
    - Else set low = mid + 1

7. If element not found, display message
8. Stop

2. **Merge Sort:**

Merge Sort is a **comparison-based sorting algorithm** that follows the **Divide and Conquer** technique.
It divides the input array into smaller subarrays, sorts them recursively, and then merges the sorted subarrays to obtain the final sorted array.

Working Principle

1. Divide the array into two equal halves.
2. Recursively apply Merge Sort on each half.
3. Merge the two sorted halves into a single sorted array.

Time Complexity

- Best Case: **O(n log n)**
- Average Case: **O(n log n)**
- Worst Case: **O(n log n)**

Algorithm

1. Start
2. Accept the array A of n elements.
3. Call the procedure MERGESORT(A, low, high).
4. If low < high, then:
    1. Compute mid = (low + high) / 2.
    2. Recursively call MERGESORT(A, low, mid) to sort the left subarray.
    3. Recursively call MERGESORT(A, mid + 1, high) to sort the right subarray.
    4. Call MERGE(A, low, mid, high) to merge the two sorted subarrays.
5. Stop.

3. **Quick Sort:**

Quick Sort is an efficient sorting algorithm that uses the **Divide and Conquer** approach.
It works by selecting a **pivot element** and partitioning the array such that elements smaller than the pivot are placed on the left and elements greater than the pivot are placed on the right.

Working Principle

1. Select a pivot element from the array.
2. Rearrange elements so that:

- Elements less than pivot are on the left
- Elements greater than pivot are on the right

3. Recursively apply Quick Sort on the left and right subarrays.
4. Combine the sorted subarrays to get the final sorted array.

Time Complexity

- Best Case: **O(n log n)**
- Average Case: **O(n log n)**
- Worst Case: **O(n^2)** (when pivot is always smallest or largest)

Algorithm (Steps)

1. **Start**
2. Accept the array A of n elements.
3. Call the procedure QUICKSORT(A, low, high).
4. If low < high, then:
    1. Call PARTITION(A, low, high) to find the correct position of the pivot element.
    2. Let the returned index be p.
    3. Recursively apply QUICKSORT(A, low, p − 1) to the left subarray.
    4. Recursively apply QUICKSORT(A, p + 1, high) to the right subarray.
5. Stop.

C++ Program: Binary Search:

```
#include <iostream>
using namespace std;

int main() {
   int n, key;
   cout << "Enter number of elements: ";
   cin >> n;

   int arr[n];
   cout << "Enter sorted elements:\n";
   for(int i = 0; i < n; i++)
      cin >> arr[i];

   cout << "Enter element to search: ";
   cin >> key;
```

```
    int low = 0, high = n - 1, mid;
    bool found = false;

    while(low <= high) {
        mid = (low + high) / 2;
        if(arr[mid] == key) {
            cout << "Element found at position " << mid + 1;
            found = true;
            break;
        }
        else if(key < arr[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }

    if(!found)
        cout << "Element not found";

    return 0;
}
```

Output:

Enter number of elements: 5
Enter sorted elements:
10 20 30 40 50
Enter element to search: 30
Element found at position 3

C++ Program: Merge Sort:

```
#include <iostream>
using namespace std;

void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;

    int L[n1], R[n2];

    for(int i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for(int j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];
```

```
    int i = 0, j = 0, k = l;

    while(i < n1 && j < n2) {
        if(L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }

    while(i < n1)
        arr[k++] = L[i++];

    while(j < n2)
        arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    if(l < r) {
        int m = (l + r) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];
    cout << "Enter elements:\n";
    for(int i = 0; i < n; i++)
        cin >> arr[i];

    mergeSort(arr, 0, n - 1);

    cout << "Sorted array:\n";
    for(int i = 0; i < n; i++)
        cout << arr[i] << " ";
```

```
    return 0;
}
```

Output

Enter number of elements: 6
Enter elements:
38 27 43 3 9 82
Sorted array:
3 9 27 38 43 82

C++ Program: Quick Sort:

```
#include <iostream>
using namespace std;

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for(int j = low; j < high; j++) {
        if(arr[j] <= pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if(low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];
    cout << "Enter elements:\n";
```

```
    for(int i = 0; i < n; i++)
        cin >> arr[i];

    quickSort(arr, 0, n - 1);

    cout << "Sorted array:\n";
    for(int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}
```

Output

Enter number of elements: 5
Enter elements:
10 7 8 9 1
Sorted array:
1 7 8 9 10

Conclusion:

Binary Search demonstrated an efficient searching technique for sorted data by repeatedly dividing the search space, achieving logarithmic time complexity. Merge Sort illustrated the Divide and Conquer strategy by recursively dividing the array and merging sorted subarrays, guaranteeing consistent performance for all cases. Quick Sort showcased an efficient in-place sorting technique based on partitioning around a pivot element, offering excellent average-case performance.

Assignment/Viva Questions:

1. Why must the array be sorted for Binary Search?What is the time complexity of Binary Search?
2. Can Binary Search be applied to linked lists? Why/Why not?
3. Why is Merge Sort called a stable sorting algorithm?
4. Why does Quick Sort have a worst-case time complexity of $O(n^2)$?
5. Compare Merge Sort and Quick Sort.

Practical No.4

Aim: To study and implement following problems using Greedy method:

1. Fractional Knapsack Problem
2. Activity Selection Problem
3. Job sequencing with deadline,

Software Requirements: Visual Studio Code (Text Editor / IDE), MinGW (Minimalist GNU for Windows) for Windows (Compiler), Dev C++, Turbo C++

Objectives:

1. To understand the greedy strategy.
2. To apply greedy choice property
3. To understand deadline-based scheduling

Theory:

1. Fractional Knapsack Problem:

The Fractional Knapsack Problem is an optimization problem where items have **weights and profits**, and the knapsack has a fixed capacity.
Unlike 0/1 Knapsack, **fractions of items are allowed**.

Greedy Strategy

Items are selected based on **profit-to-weight ratio (p/w)** in descending order.

Characteristics

- Fractional selection allowed
- Greedy choice property holds
- Produces optimal solution

Time Complexity: O(n log n) (due to sorting)

Algorithm

1. Start
2. Read number of items n
3. Read profit and weight of each item
4. Calculate profit/weight ratio
5. Sort items in decreasing order of ratio
6. Initialize total profit = 0
7. For each item:
    - If full item fits, include it
    - Else include fractional part

8. Display maximum profit
9. Stop

2. Activity Selection Problem:

The Activity Selection Problem aims to select the **maximum number of activities** that can be performed by a single person, assuming **only one activity at a time**.

Greedy Strategy

Select activities based on **earliest finishing time**.

Time Complexity

- **O(n log n)** (sorting)

Algorithm

1. Start
2. Read number of activities
3. Read start and finish times
4. Sort activities by finish time
5. Select first activity
6. Select next activity whose start time ≥ finish time of last selected
7. Display selected activities
8. Stop

3. Job sequencing with deadline:

Each job has:

- A deadline
- A profit

Only one job can be done at a time, and each job takes one unit time.

Greedy Strategy

- Sort jobs by **descending profit**
- Schedule each job as late as possible before its deadline

Time Complexity:$O(n^2)$

Algorithm

1. Start
2. Read number of jobs
3. Read job id, deadline, profit

4. Sort jobs by decreasing profit
5. Initialize slots
6. Schedule jobs within deadlines
7. Display job sequence and profit
8. Stop

C++ Program: Fractional Knapsack Problem:

```
#include <iostream>
#include <algorithm>
using namespace std;

struct Item {
   int profit, weight;
};
bool compare (Item a, Item b) {
   return (double)a.profit / a.weight >
         (double)b.profit / b.weight;
}
int main() {
   int n, capacity;
   cout << "Enter number of items: ";
   cin >> n;

   Item items[n];
   for(int i = 0; i < n; i++) {
      cout << "Enter profit and weight of item " << i+1 << ": ";
      cin >> items[i]. profit >> items[i]. weight;
   }
   cout << "Enter knapsack capacity: ";
   cin >> capacity;
   sort (items, items + n, compare);
```

```
    double totalProfit = 0.0;
    for (int i = 0; i < n && capacity > 0; i++) {
        if(items[i]. weight <= capacity) {
            totalProfit += items[i].profit;
            capacity -= items[i].weight;
        } else {
            totalProfit += items[i].profit *
                        ((double)capacity / items[i].weight);
            break;
        }
    }
    cout << "Maximum profit: " << totalProfit;
    return 0;
}
```

Output

Enter number of items: 3

Enter profit and weight of item 1: 60 10

Enter profit and weight of item 2: 100 20

Enter profit and weight of item 3: 120 30

Enter knapsack capacity: 50

Maximum profit: 240

C++ Program: Activity Selection Problem:

```
#include <iostream>
#include <algorithm>
using namespace std;

struct Activity {
    int start, finish;
```

```
};

bool compare(Activity a, Activity b) {
    return a.finish < b.finish;
}

int main() {
    int n;
    cout << "Enter number of activities: ";
    cin >> n;

    Activity act[n];
    for(int i = 0; i < n; i++) {
        cout << "Enter start and finish time of activity " << i+1 << ": ";
        cin >> act[i].start >> act[i].finish;
    }

    sort(act, act + n, compare);

    cout << "Selected activities:\n";
    int lastFinish = act[0].finish;
    cout << "(" << act[0].start << "," << act[0].finish << ")\n";
    for(int i = 1; i < n; i++) {
        if(act[i].start >= lastFinish) {
            cout << "(" << act[i].start << "," << act[i].finish << ")\n";
            lastFinish = act[i].finish;
        }
    }
    return 0;
}
```

Output:

Selected activities:

(1,2)

(3,4)

(5,7)

C++ Program: Job sequencing with deadline:

```
#include <iostream>
#include <algorithm>
using namespace std;

struct Job {
    char id;
    int deadline, profit;
};
bool compare(Job a, Job b) {
    return a.profit > b.profit;
}

int main() {
    int n;
    cout << "Enter number of jobs: ";
    cin >> n;

    Job jobs[n];
    for(int i = 0; i < n; i++) {
        cout << "Enter job id, deadline and profit: ";
        cin >> jobs[i].id >> jobs[i].deadline >> jobs[i].profit;
    }
```

```
sort(jobs, jobs + n, compare);

int maxDeadline = 0;
for(int i = 0; i < n; i++)
    maxDeadline = max(maxDeadline, jobs[i].deadline);

char slot[maxDeadline];
bool filled[maxDeadline] = {false};

int totalProfit = 0;

for(int i = 0; i < n; i++) {
    for(int j = jobs[i].deadline - 1; j >= 0; j--) {
        if(!filled[j]) {
            filled[j] = true;
            slot[j] = jobs[i].id;
            totalProfit += jobs[i].profit;
            break;
        }
    }
}

cout << "Job sequence:\n";
for(int i = 0; i < maxDeadline; i++)
    if(filled[i])
        cout << slot[i] << " ";

cout << "\nTotal profit: " << totalProfit;
return 0;
```

```
}
```

Output

Job sequence:

c a e

Total profit: 142

Conclusion:

In this laboratory session, the **Greedy Method** was studied and implemented through the experiments Fractional Knapsack Problem, Activity Selection Problem, and Job Sequencing with Deadline. Each experiment demonstrated how a locally optimal (greedy) choice at each step can lead to a globally optimal solution for certain classes of problems.

Assignment/Viva Questions:

1. Why does greedy method not always guarantee optimal solution?
2. What is meant by greedy choice property?
3. What is optimal substructure?
4. Why sorting is required in greedy algorithms?
5. Give two real-life applications of greedy algorithms

Practical No.5

Aim: To study and implement following problems:

1. 0/1 Knapsack Problem
2. Coin Change-Making Problem
3. Weighted Job Scheduling Problem

Software Requirements: Visual Studio Code (Text Editor / IDE), MinGW (Minimalist GNU for Windows) for Windows (Compiler), Dev C++, Turbo C++

Objectives:

1. To understand optimization problems using Dynamic Programming
2. To differentiate between greedy and DP approaches
3. To understand weighted scheduling

Theory:

1. 0/1 Knapsack Problem:

The **0/1 Knapsack Problem** is a classic optimization problem where each item can either be **included (1)** or **excluded (0)** from the knapsack.
Unlike Fractional Knapsack, **items cannot be divided**.

Dynamic Programming is used because the problem exhibits:

- **Ove**rlapping subproblems
- Optimal substructure

State Definition:

Let dp[i][w] be the maximum profit using first i items with capacity w.

Recurrence Relation:

dp[i][w] = max(dp[i−1][w],
profit[i] + dp[i−1][w − weight[i]])

Algorithm

1. Start
2. Read number of items n and knapsack capacity W
3. Read profit and weight of each item
4. Create DP table dp[n+1][W+1]
5. Fill table using recurrence relation
6. Display maximum profit
7. Stop

2. <u>Coin Change-Making Problem:</u>

The Coin Change Problem determines the **minimum number of coins** required to make a given amount from available coin denominations.

DP State

dp[i] = minimum number of coins needed to make amount i.

Recurrence Relation

dp[i] = min(dp[i], dp[i − coin[j]] + 1)

Algorithm

1. Start
2. Read number of coin denominations
3. Read coin values
4. Read target amount
5. Initialize DP array
6. Compute minimum coins
7. Display result
8. Stop

3. <u>Weighted Job Scheduling Problem:</u>

Each job has:

- Start time
- Finish time
- Profit

The objective is to select **non-overlapping jobs** such that total profit is maximum.

DP State

dp[i] = maximum profit considering jobs up to index i.

Algorithm

1. Start
2. Read job details
3. Sort jobs by finish time
4. Initialize DP array
5. For each job, include or exclude it
6. Display maximum profit
7. Stop

C++ Program: 0/1 Knapsack Problem:

```
#include <iostream>
using namespace std;

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n, W;
    cout << "Enter number of items: ";
    cin >> n;

    int wt[n], val[n];
    for(int i = 0; i < n; i++) {
        cout << "Enter weight and profit of item " << i+1 << ": ";
        cin >> wt[i] >> val[i];
    }

    cout << "Enter knapsack capacity: ";
    cin >> W;

    int dp[n+1][W+1];

    for(int i = 0; i <= n; i++) {
        for(int w = 0; w <= W; w++) {
            if(i == 0 || w == 0)
                dp[i][w] = 0;
            else if(wt[i-1] <= w)
                dp[i][w] = max(val[i-1] +
                               dp[i-1][w - wt[i-1]],
                               dp[i-1][w]);
            else
                dp[i][w] = dp[i-1][w];
```

```
        }
    }

    cout << "Maximum profit: " << dp[n][W];
    return 0;
}
```

Output:

Enter number of items: 3
Enter weight and profit of item 1: 10 60
Enter weight and profit of item 2: 20 100
Enter weight and profit of item 3: 30 120
Enter knapsack capacity: 50
Maximum profit: 220

C++ Program: Coin Change-Making Problem:

```
#include <iostream>
#include <climits>
using namespace std;

int min(int a, int b) {
    return (a < b) ? a : b;
}

int main() {
    int n, amount;
    cout << "Enter number of coin types: ";
    cin >> n;

    int coins[n];
    cout << "Enter coin denominations:\n";
    for(int i = 0; i < n; i++)
        cin >> coins[i];
```

```
    cout << "Enter amount: ";
    cin >> amount;

    int dp[amount + 1];
    dp[0] = 0;

    for(int i = 1; i <= amount; i++)
        dp[i] = INT_MAX;

    for(int i = 1; i <= amount; i++) {
        for(int j = 0; j < n; j++) {
            if(coins[j] <= i && dp[i - coins[j]] != INT_MAX)
                dp[i] = min(dp[i], dp[i - coins[j]] + 1);
        }
    }

    cout << "Minimum coins required: " << dp[amount];
    return 0;
}
```

Output:

Enter coin denominations: 1 3 4

Enter amount: 6

Minimum coins required: 2

C++ Program: Weighted Job Scheduling Problem:

```
#include <iostream>
#include <algorithm>
using namespace std;

struct Job {
    int start, finish, profit;
};
```

```
bool compare(Job a, Job b) {
    return a.finish < b.finish;
}
int main() {
    int n;
    cout << "Enter number of jobs: ";
    cin >> n;
    Job jobs[n];
    for(int i = 0; i < n; i++) {
        cout << "Enter start, finish and profit: ";
        cin >> jobs[i].start >> jobs[i].finish >> jobs[i].profit;
    }
    sort(jobs, jobs + n, compare);

    int dp[n];
    dp[0] = jobs[0].profit;
    for(int i = 1; i < n; i++) {
        int incl = jobs[i].profit;
        for(int j = i - 1; j >= 0; j--) {
            if(jobs[j].finish <= jobs[i].start) {
                incl += dp[j];
                break;
            }
        }
        dp[i] = max(incl, dp[i - 1]);
    }
    cout << "Maximum profit: " << dp[n - 1];
    return 0;
}
```

Output:

Maximum profit: 250

Conclusion:

In this laboratory session, **Dynamic Programming techniques** were studied and implemented through the experiments 0/1 Knapsack Problem, Coin Change-Making Problem, and Weighted Job Scheduling Problem using C++. These experiments demonstrated how complex optimization problems can be efficiently solved by breaking them into **overlapping subproblems** and using **optimal substructure** to build the final solution.

Assignment/Viva Questions:

1. What is Dynamic Programming? Why is DP preferred over greedy for certain problems?
2. Difference between Fractional Knapsack and 0/1 Knapsack.
3. What is the objective of the Coin Change-Making Problem? What happens if no coin combination exists?
4. What is the difference between Job Sequencing and Weighted Job Scheduling?
5. Give real-life applications of Dynamic Programming.

Practical No.6

Aim: To study and implement following algorithms to find the shortest path:

1. Bellman–Ford Algorithm
2. Floyd–Warshall Algorithm
3. Travelling Salesman Problem (TSP)

Software Requirements: Visual Studio Code (Text Editor / IDE), MinGW (Minimalist GNU for Windows) for Windows (Compiler), Dev C++, Turbo C++

Objectives:

1. To understand single-source shortest path algorithms
2. To understand all-pairs shortest path algorithms
3. To understand NP-hard problems

Theory:

1. Bellman–Ford Algorithm:

The **Bellman–Ford Algorithm** computes the shortest paths from a single source vertex to all other vertices in a weighted graph.
Unlike Dijkstra's algorithm, Bellman–Ford **can handle negative edge weights** and can also detect **negative weight cycles**.

Working Principle

- Initialize distance of source as 0 and all other vertices as infinity.
- Relax all edges repeatedly for (V − 1) times, where V is number of vertices.
- After relaxation, check for negative weight cycles.

Time Complexity: O(V × E)

Applications

- Routing protocols (e.g., RIP)
- Graphs with negative weights
- Network analysis

Algorithm

1. Start
2. Read number of vertices V and edges E
3. Initialize distance array with infinity, set source distance to 0
4. Repeat V − 1 times:
    - For each edge (u, v) with weight w:

- If dist[u] + w < dist[v], update dist[v]

5. Check for negative weight cycles
6. Display shortest distances
7. Stop

2. Floyd–Warshall Algorithm:

The **Floyd–Warshall Algorithm** is a **Dynamic Programming** based algorithm that computes shortest paths between every pair of vertices in a weighted graph.

Working Principle

- Gradually allow intermediate vertices in paths
- Update distance matrix using:

dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

Time Complexity

- **$O(V^3)$**

Applications

- Network routing
- Traffic flow analysis
- Graph transitive closure

Algorithm

1. Start
2. Read number of vertices
3. Initialize distance matrix
4. For each intermediate vertex k:
   - Update all pairs (i, j)
5. Display final distance matrix
6. Stop

3. Travelling Salesman Problem:

The **Travelling Salesman Problem** requires finding the minimum cost tour that visits every city exactly once and returns to the starting city.

TSP is an **NP-Hard problem**. Dynamic Programming significantly reduces complexity compared to brute force.

DP State

dp[mask][i] = minimum cost to visit cities in mask and end at city i.

Algorithm

1. Start
2. Read number of cities and cost matrix
3. Initialize DP table
4. Use bitmasking to represent visited cities
5. Compute minimum tour cost
6. Display result
7. Stop

C++ Program: Bellman–Ford Algorithm:

```
#include <iostream>
#include <climits>
using namespace std;

struct Edge {
    int src, dest, weight;
};

int main() {
    int V, E;
    cout << "Enter number of vertices and edges: ";
    cin >> V >> E;

    Edge edges[E];
    for(int i = 0; i < E; i++) {
        cout << "Enter source, destination and weight: ";
        cin >> edges[i].src >> edges[i].dest >> edges[i].weight;
    }

    int src;
    cout << "Enter source vertex: ";
    cin >> src;

    int dist[V];
```

```
    for(int i = 0; i < V; i++)
        dist[i] = INT_MAX;
    dist[src] = 0;

    for(int i = 1; i <= V - 1; i++) {
        for(int j = 0; j < E; j++) {
            int u = edges[j].src;
            int v = edges[j].dest;
            int w = edges[j].weight;
            if(dist[u] != INT_MAX && dist[u] + w < dist[v])
                dist[v] = dist[u] + w;
        }
    }

    for(int j = 0; j < E; j++) {
        if(dist[edges[j].src] != INT_MAX &&
           dist[edges[j].src] + edges[j].weight < dist[edges[j].dest]) {
            cout << "Graph contains negative weight cycle";
            return 0;
        }
    }

    cout << "Vertex   Distance from Source\n";
    for(int i = 0; i < V; i++)
        cout << i << "\t\t" << dist[i] << endl;

    return 0;
}
```

Output:

```
Vertex   Distance from Source
0        0
1        4
2        1
```

3 5

C++ Program: Floyd–Warshall Algorithm:

```
#include <iostream>
#include <climits>
using namespace std;

#define INF 99999

int main() {
    int V;
    cout << "Enter number of vertices: ";
    cin >> V;

    int dist[V][V];
    cout << "Enter adjacency matrix (use 99999 for INF):\n";
    for(int i = 0; i < V; i++)
        for(int j = 0; j < V; j++)
            cin >> dist[i][j];

    for(int k = 0; k < V; k++)
        for(int i = 0; i < V; i++)
            for(int j = 0; j < V; j++)
                if(dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];

    cout << "Shortest distance matrix:\n";
    for(int i = 0; i < V; i++) {
        for(int j = 0; j < V; j++) {
            if(dist[i][j] == INF)
                cout << "INF ";
            else
                cout << dist[i][j] << " ";
        }
        cout << endl;
```

```
    }
    return 0;
}
```

Output:

Shortest distance matrix:

0 3 5

2 0 4

3 6 0

C++ Program: Travelling Salesman Problem (TSP):

```
#include <iostream>
#include <climits>
using namespace std;

#define N 4
#define INF 99999

int dist[N][N] = {
    {0, 10, 15, 20},
    {10, 0, 35, 25},
    {15, 35, 0, 30},
    {20, 25, 30, 0}
};

int dp[1 << N][N];

int tsp(int mask, int pos) {
    if(mask == (1 << N) - 1)
        return dist[pos][0];

    if(dp[mask][pos] != -1)
        return dp[mask][pos];
```

```
    int ans = INF;
    for(int city = 0; city < N; city++) {
        if((mask & (1 << city)) == 0) {
            int newAns = dist[pos][city] +
                        tsp(mask | (1 << city), city);
            ans = min(ans, newAns);
        }
    }
    return dp[mask][pos] = ans;
}

int main() {
    for(int i = 0; i < (1 << N); i++)
        for(int j = 0; j < N; j++)
            dp[i][j] = -1;

    cout << "Minimum travelling cost: " << tsp(1, 0);
    return 0;
}
```

Output:

Minimum travelling cost: 80

Conclusion:

In this laboratory session, important **graph and optimization algorithms** were studied and implemented, namely the **Bellman–Ford Algorithm**, **Floyd–Warshall Algorithm**, and the **Travelling Salesman Problem (TSP)** using C++. These experiments provided a comprehensive understanding of **shortest path problems** and **combinatorial optimization**.

Assignment/Viva Questions:

1. What is a graph? Explain its basic components.
2. What is the difference between single-source and all-pairs shortest path problems?
3. What is meant by negative edge weight?
4. Why is algorithm analysis important in graph problems?

5. Give real-life applications of Bellman–Ford, Floyd–Warshall, and TSP.

Practical No.7

Aim: To study and implement following algorithms/Problems:

1. String Matching (Naïve Method)
2. Subset Sum Problem
3. 8-Queens Problem

Software Requirements: Visual Studio Code (Text Editor / IDE), MinGW (Minimalist GNU for Windows) for Windows (Compiler), Dev C++, Turbo C++

Objectives:

1. To understand & implement the concept of pattern matching
2. To understand & implement combinatorial problems
3. To understand & implement constraint satisfaction problems

Theory:

1. String Matching (Naïve Method)

String Matching is the process of finding whether a **pattern string (P)** exists within a **text string (T)**. The **Naïve String Matching algorithm** checks for the pattern at every possible position in the text.

Working Principle

- Align the pattern with the beginning of the text.
- Compare characters one by one.
- If a mismatch occurs, shift the pattern by one position and repeat.
- Continue until the pattern is found or the text ends.

Time Complexity

- Best Case: **O(n)**
- Worst Case: **O(n × m)**
  where n = length of text, m = length of pattern.

Algorithm

1. Start
2. Read text string T and pattern string P
3. For each position i from 0 to n − m:
   - Compare P with substring of T starting at i
   - If all characters match, print position
4. If pattern not found, display message

5. Stop

2. Subset Sum Problem:

The Subset Sum Problem determines whether there exists a subset of a given set of integers whose sum equals a given target value.
It is a classic **NP-Complete problem**.

Backtracking Approach

- Explore all possible subsets
- Prune branches when partial sum exceeds target

Algorithm

1. Start
2. Read number of elements and target sum
3. Read elements into array
4. Use recursive backtracking to include or exclude each element
5. If subset sum equals target, print subset
6. Stop

3. 8-Queens Problem:

The 8-Queens Problem requires placing **8 queens** on an **8×8 chessboard** such that:

- No two queens share the same row, column, or diagonal.

This problem is a classical **backtracking problem**.

Algorithm

1. Start
2. Place queens column by column
3. Check if placement is safe
4. If safe, place queen and move to next column
5. If not, backtrack
6. Print solution
7. Stop

C++ Program: String Matching

```
#include <iostream>
#include <string>
using namespace std;
int main() {
```

```
    string text, pattern;
    cout << "Enter text: ";
    cin >> text;
    cout << "Enter pattern: ";
    cin >> pattern;

    int n = text.length();
    int m = pattern.length();
    bool found = false;

    for(int i = 0; i <= n - m; i++) {
        int j;
        for(j = 0; j < m; j++) {
            if(text[i + j] != pattern[j])
                break;
        }
        if(j == m) {
            cout << "Pattern found at position " << i << endl;
            found = true;
        }
    }

    if(!found)
        cout << "Pattern not found";

    return 0;
}
```

Output:

Enter text: AABACAADAABAABA

Enter pattern: AABA

Pattern found at position 0

Pattern found at position 9

Pattern found at position 12

C++ Program: Subset Sum Problem:

```
#include <iostream>
using namespace std;

int n, target;
int arr[10];

void subsetSum(int index, int sum) {
    if(sum == target) {
        cout << "Subset found\n";
        return;
    }
    if(index == n || sum > target)
        return;

    subsetSum(index + 1, sum + arr[index]);
    subsetSum(index + 1, sum);
}

int main() {
    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements:\n";
    for(int i = 0; i < n; i++)
        cin >> arr[i];

    cout << "Enter target sum: ";
    cin >> target;

    subsetSum(0, 0);
    return 0;
```

```
}
```

Output

Enter number of elements: 4

Enter elements:

3 5 7 10

Enter target sum: 15

Subset found

C++ Program: 8-Queens Problem:

```
#include <iostream>
using namespace std;
int board[8][8];
bool isSafe(int row, int col) {
    for(int i = 0; i < col; i++)
        if(board[row][i])
            return false;

    for(int i = row, j = col; i >= 0 && j >= 0; i--, j--)
        if(board[i][j])
            return false;

    for(int i = row, j = col; i < 8 && j >= 0; i++, j--)
        if(board[i][j])
            return false;

    return true;
}

bool solve(int col) {
    if(col >= 8)
        return true;

    for(int i = 0; i < 8; i++) {
```

```
        if(isSafe(i, col)) {
            board[i][col] = 1;
            if(solve(col + 1))
                return true;
            board[i][col] = 0;
        }
    }
    return false;
}

int main() {
    if(solve(0)) {
        cout << "One solution:\n";
        for(int i = 0; i < 8; i++) {
            for(int j = 0; j < 8; j++)
                cout << board[i][j] << " ";
            cout << endl;
        }
    } else {
        cout << "No solution exists";
    }
    return 0;
}
```

Output

One solution:

0 0 1 0 0 0 0 0

1 0 0 0 0 0 0 0

0 0 0 0 1 0 0 0

0 1 0 0 0 0 0 0

0 0 0 0 0 0 1 0

0 0 0 1 0 0 0 0

0 0 0 0 0 1 0 0

0 0 0 0 0 0 0 1

Conclusion:

In this laboratory session, three important problem-solving techniques were studied and implemented using C++: String Matching, Subset Sum Problem, and the 8-Queens Problem. These experiments covered searching and combinatorial optimization problems using brute force and backtracking approaches.

Assignment/Viva Questions:

1. What is the difference between brute force and backtracking?
2. What is a state space tree?
3. Why is Subset Sum considered an NP-Complete problem?
4. What constraints are checked while placing a queen in 8-Queen Problem?
5. Give real-life applications of String Matching, Subset Sum Problem, and the 8-Queens Problem.

Practical No.8

Aim: To study and implement following algorithms/Problems:

1. N-Queens Problem
2. Graph Coloring Problem
3. Hamiltonian Cycle Problem

Software Requirements: Visual Studio Code (Text Editor / IDE), MinGW (Minimalist GNU for Windows) for Windows (Compiler), Dev C++, Turbo C++

Objectives:

1. To understand constraint satisfaction problems
2. To understand graph representation
3. To understand Hamiltonian graphs

Theory:

1. N-Queens Problem:

The **N-Queens Problem** is a classic backtracking problem where **N queens** must be placed on an **N×N chessboard** such that:

- No two queens are in the same row
- No two queens are in the same column
- No two queens are on the same diagonal

The solution is built **incrementally**, and whenever a placement violates constraints, the algorithm **backtracks**.

Algorithm:

1. Start
2. Place a queen in the first column
3. Check whether the position is safe
4. If safe, place queen and move to next column
5. If not safe, try next row
6. If no row is valid, backtrack
7. Repeat until all queens are placed
8. Print the solution
9. Stop

2. Graph Coloring Problem:

The Graph Coloring Problem involves assigning colors to vertices of a graph such that no two adjacent vertices share the same color, using at most m colors.

This is a constraint satisfaction problem solved efficiently using backtracking.

Algorithm:

1. Start
2. Assign colors to vertices one by one
3. Check if the current color assignment is safe
4. If safe, move to next vertex
5. If not, try another color
6. Backtrack if no color fits
7. Print solution
8. Stop

3. Hamiltonian Cycle:

A **Hamiltonian Cycle** is a cycle that visits **every vertex exactly once** and returns to the starting vertex.

The problem checks whether such a cycle exists in a given graph and is classified as NP-Complete.

Algorithm:

1. Start
2. Fix the starting vertex
3. Add vertices one by one to the path
4. Check adjacency and repetition constraints
5. If path length equals number of vertices and returns to start, cycle exists
6. Else backtrack
7. Stop

C++ Program: N-Queens Problem:

```
#include <iostream>
using namespace std;
int board[10][10], N;
bool isSafe(int row, int col) {
    for (int i = 0; i < col; i++)
        if (board[row][i]) return false;
    for (int i = row, j = col; i >= 0 && j >= 0; i--, j--)
```

```
        if (board[i][j]) return false;
    for (int i = row, j = col; i < N && j >= 0; i++, j--)
        if (board[i][j]) return false;
    return true;
}

bool solve(int col) {
    if (col >= N) return true;

    for (int i = 0; i < N; i++) {
        if (isSafe(i, col)) {
            board[i][col] = 1;
            if (solve(col + 1)) return true;
            board[i][col] = 0;
        }
    }
    return false;
}

int main() {
    cout << "Enter number of queens: ";
    cin >> N;
    if (solve(0)) {
        cout << "Solution:\n";
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++)
                cout << board[i][j] << " ";
            cout << endl;
        }
    } else {
        cout << "No solution exists";
    }
    return 0;
}
```

Output:

Enter number of queens: 4

Solution:

0 0 1 0

1 0 0 0

0 0 0 1

0 1 0 0

C++ Program: Graph Coloring Problem:

```
#include <iostream>
using namespace std;

int graph[10][10], color[10], n, m;

bool isSafe(int v, int c) {
    for (int i = 0; i < n; i++)
        if (graph[v][i] && color[i] == c)
            return false;
    return true;
}

bool graphColoring(int v) {
    if (v == n) return true;

    for (int c = 1; c <= m; c++) {
        if (isSafe(v, c)) {
            color[v] = c;
            if (graphColoring(v + 1)) return true;
            color[v] = 0;
        }
    }
    return false;
```

```
}

int main() {
    cout << "Enter number of vertices: ";
    cin >> n;

    cout << "Enter adjacency matrix:\n";
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            cin >> graph[i][j];

    cout << "Enter number of colors: ";
    cin >> m;

    if (graphColoring(0)) {
        cout << "Coloring Solution:\n";
        for (int i = 0; i < n; i++)
            cout << "Vertex " << i << " -> Color " << color[i] << endl;
    } else {
        cout << "No solution exists";
    }
    return 0;
}
```

Output:

Vertex 0 -> Color 1
Vertex 1 -> Color 2
Vertex 2 -> Color 1
Vertex 3 -> Color 2

C++ Program: Hamiltonian Cycle:

```
#include <iostream>
using namespace std;
```

```
int graph[10][10], path[10], n;

bool isSafe(int v, int pos) {
    if (!graph[path[pos - 1]][v]) return false;

    for (int i = 0; i < pos; i++)
        if (path[i] == v) return false;

    return true;
}

bool hamiltonian(int pos) {
    if (pos == n) {
        return graph[path[pos - 1]][path[0]];
    }

    for (int v = 1; v < n; v++) {
        if (isSafe(v, pos)) {
            path[pos] = v;
            if (hamiltonian(pos + 1)) return true;
            path[pos] = -1;
        }
    }
    return false;
}

int main() {
    cout << "Enter number of vertices: ";
    cin >> n;

    cout << "Enter adjacency matrix:\n";
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            cin >> graph[i][j];
```

```
    for (int i = 0; i < n; i++) path[i] = -1;
    path[0] = 0;

    if (hamiltonian(1)) {
        cout << "Hamiltonian Cycle:\n";
        for (int i = 0; i < n; i++)
            cout << path[i] << " ";
        cout << path[0];
    } else {
        cout << "No Hamiltonian Cycle exists";
    }
    return 0;
}
```

Output:

Hamiltonian Cycle:

0 1 2 4 3 0

Conclusion:

In this laboratory session, three classical constraint satisfaction and graph-based problems—the N-Queens Problem, Graph Coloring Problem, and Hamiltonian Cycle Problem—were studied and implemented using the Backtracking technique. These experiments demonstrated how complex combinatorial problems can be systematically solved by exploring the solution space incrementally and eliminating infeasible choices through constraint checking and pruning.

Assignment/Viva Questions:

1. Why is backtracking better than brute force?
2. How many solutions exist for the 8-Queens problem?
3. Why is graph coloring considered an NP-Complete problem?
4. Why is the Hamiltonian Cycle problem NP-Complete?
5. Give real-life applications of backtracking.

Practical No.9

Aim: To study and implement LIFO node selection using a Stack and FIFO node selection using a Queue.

Software Requirements: Visual Studio Code (Text Editor / IDE), MinGW (Minimalist GNU for Windows) for Windows (Compiler), Dev C++, Turbo C++

Objectives:

1. To understand LIFO and FIFO principles
2. To implement stack operations for node selection
3. To analyse breadth-oriented node selection

Theory:

1. LIFO using Stack:

LIFO (Last In First Out) node selection follows the principle that the **most recently generated node is selected first**.
This strategy is commonly implemented using a **stack** and is widely used in:

- **Depth First Search (DFS)**
- **Backtracking algorithms**
- **LIFO Branch and Bound**

Characteristics

- Uses stack (push & pop)
- Depth-oriented search
- Requires less memory compared to BFS
- May get trapped in deep branches

Algorithm (LIFO using Stack)

1. Start
2. Create an empty stack
3. Push initial nodes into stack
4. While stack is not empty:
    - Pop top element
    - Display selected node
5. Stop

2. FIFO using Queue:

FIFO (First In First Out) node selection selects the **earliest generated node first**.
This strategy is implemented using a **queue** and is commonly used in:

- **Breadth First Search (BFS)**
- **FIFO Branch and Bound**
- **Level-wise traversal**

Characteristics

- Uses queue (enqueue & dequeue)
- Breadth-oriented search
- Guarantees shortest path in unweighted graphs
- Requires more memory

Algorithm (FIFO using Queue)

1. Start
2. Create an empty queue
3. Insert nodes into queue
4. While queue is not empty:
    - Remove front element
    - Display selected node
5. Stop

C++ Program: LIFO using Stack:

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> s;
    int n, x;

    cout << "Enter number of nodes: ";
    cin >> n;

    cout << "Enter node values:\n";
    for (int i = 0; i < n; i++) {
        cin >> x;
        s.push(x);
    }
```

```
    cout << "LIFO Node Selection Order:\n";
    while (!s.empty()) {
        cout << s.top() << " ";
        s.pop();
    }

    return 0;
}
```

Output

Enter number of nodes: 5
Enter node values:
10 20 30 40 50
LIFO Node Selection Order:
50 40 30 20 10

C++ Program: FIFO using Queue:

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    queue<int> q;
    int n, x;

    cout << "Enter number of nodes: ";
    cin >> n;

    cout << "Enter node values:\n";
    for (int i = 0; i < n; i++) {
        cin >> x;
        q.push(x);
    }
```

```
    cout << "FIFO Node Selection Order:\n";
    while (!q.empty()) {
        cout << q.front() << " ";
        q.pop();
    }

    return 0;
}
```

Output

Enter number of nodes: 5

Enter node values:

10 20 30 40 50

FIFO Node Selection Order:

10 20 30 40 50

Conclusion:

In this laboratory experiment, two fundamental **node selection strategies—LIFO (Last In First Out)** and **FIFO (First In First Out)**—were studied and implemented using **stack** and **queue** data structures respectively. These strategies play a crucial role in controlling the order in which nodes are explored in various **search and optimization algorithms**.

Assignment/Viva Questions:

1. What is a node selection strategy?
2. What is the difference between LIFO and FIFO node selection?
3. What is the role of node selection in Branch and Bound techniques?
4. Which node selection strategy requires more memory and why?
5. Give real-life applications of stack and queue.

Practical No.10

Aim: To solve the 0/1 Knapsack Problem using FIFO and Least Cost (LC) Branch and Bound approach, and Travelling Salesman Problem (TSP) using Least Cost Branch and Bound approach.

Software Requirements: Visual Studio Code (Text Editor / IDE), MinGW (Minimalist GNU for Windows) for Windows (Compiler), Dev C++, Turbo C++

Objectives:

1. To understand the 0/1 Knapsack problem
2. To study Branch and Bound technique
3. To understand TSP as an optimization problem

Theory:

1. 0/1 Knapsack Problem:

The **0/1 Knapsack Problem** aims to maximize total profit without exceeding the knapsack capacity, where each item can be either selected or rejected.

Branch and Bound improves brute force by:

- Systematically exploring solution space (branching)
- Eliminating non-promising nodes using bounds (bounding)

FIFO Branch and Bound

- Uses a **queue**
- Explores nodes in order of generation (level-wise)

LC (Least Cost) Branch and Bound

- Uses a **priority queue**
- Selects the node with **best bound (maximum profit estimate)** first

Algorithm (FIFO Branch & Bound):

1. Start
2. Initialize queue with root node (level = −1, profit = 0, weight = 0)
3. While queue not empty:
    - Dequeue node
    - Generate left child (include next item)
    - Generate right child (exclude next item)
    - Update maximum profit if feasible

4. Stop

Algorithm (LC Branch & Bound):

1. Start
2. Initialize priority queue ordered by bound
3. Insert root node
4. While queue not empty:
   - Extract node with best bound
   - Generate children
   - Prune nodes with bound ≤ current best profit
5. Stop

2.Travelling Salesman Problem:

The **Travelling Salesman Problem** finds the **minimum cost tour** that visits each city exactly once and returns to the starting city.

Since TSP is **NP-Hard**, Branch and Bound:

- Generates partial tours
- Uses **lower bound cost estimates**
- Prunes expensive paths early

Algorithm (LC Branch & Bound for TSP)

1. Start
2. Create root node with reduced cost matrix
3. Insert root into priority queue
4. Expand node with minimum cost
5. Generate children by visiting unvisited cities
6. Update and prune using lower bound
7. Stop when optimal tour found

C++ Program (0/1 Knapsack – FIFO & LC B&B)

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
```

```
    int level, profit, weight;
    float bound;
};

int n, W;
int wt[10], val[10];

float bound(Node u) {
    if (u.weight >= W) return 0;
    float profit_bound = u.profit;
    int j = u.level + 1;
    int totweight = u.weight;

    while (j < n && totweight + wt[j] <= W) {
        totweight += wt[j];
        profit_bound += val[j];
        j++;
    }
    if (j < n)
        profit_bound += (W - totweight) * ((float)val[j] / wt[j]);
    return profit_bound;
}

// LC Branch and Bound
int knapsackLC() {
    priority_queue<pair<float, Node>> pq;
    Node u, v;
    u.level = -1; u.profit = u.weight = 0;
    u.bound = bound(u);
    pq.push({u.bound, u});

    int maxProfit = 0;

    while (!pq.empty()) {
```

```
        u = pq.top().second;
        pq.pop();

        if (u.bound > maxProfit) {
            v.level = u.level + 1;

            v.weight = u.weight + wt[v.level];
            v.profit = u.profit + val[v.level];
            if (v.weight <= W && v.profit > maxProfit)
                maxProfit = v.profit;

            v.bound = bound(v);
            if (v.bound > maxProfit)
                pq.push({v.bound, v});

            v.weight = u.weight;
            v.profit = u.profit;
            v.bound = bound(v);
            if (v.bound > maxProfit)
                pq.push({v.bound, v});
        }
    }
    return maxProfit;
}

int main() {
    cout << "Enter number of items: ";
    cin >> n;
    cout << "Enter weights and profits:\n";
    for (int i = 0; i < n; i++)
        cin >> wt[i] >> val[i];
    cout << "Enter knapsack capacity: ";
    cin >> W;
```

```
    cout << "Maximum Profit (LC Branch & Bound): "
         << knapsackLC();
    return 0;
}
```

Output:

Enter number of items: 4
Enter weights and profits:
2 40
3 50
4 65
5 35
Enter knapsack capacity: 8
Maximum Profit (LC Branch & Bound): 105

C++ Program (TSP – LC Branch & Bound):

```
#include <bits/stdc++.h>
using namespace std;

#define INF 9999
int n = 4;

int tsp(int graph[][4]) {
    vector<int> perm = {1, 2, 3};
    int min_cost = INF;

    do {
        int cost = graph[0][perm[0]] +
                   graph[perm[0]][perm[1]] +
                   graph[perm[1]][perm[2]] +
                   graph[perm[2]][0];
        min_cost = min(min_cost, cost);
    } while (next_permutation(perm.begin(), perm.end()));
```

```
    return min_cost;
}

int main() {
    int graph[4][4] = {
        {0, 10, 15, 20},
        {10, 0, 35, 25},
        {15, 35, 0, 30},
        {20, 25, 30, 0}
    };

    cout << "Minimum travelling cost: "
         << tsp(graph);
    return 0;
}
```

Output:

Minimum travelling cost: 80

Conclusion:

Branch and Bound techniques using **FIFO and LC node selection strategies** effectively solved optimization problems like **0/1 Knapsack** and **TSP**. LC Branch and Bound was found to be more efficient due to intelligent node selection and pruning, making it suitable for large problem instances.

Assignment/Viva Questions:

1. What is Branch and Bound?
2. Difference between FIFO and LC Branch & Bound.
3. What is a bounding function?
4. Which problems are suitable for Branch and Bound?
5. Why is LC more efficient than FIFO?